Efficient Zero-Shot Argument Mining of UN Resolutions with Small Open-Weight LMs and Semantic Pre-Filtering

Authors:

Siddhartha Khandelwal
GLA University
Mathura, UP, India
siddharthakhandelwal9@gmail.com

Jyotsana Bhardwaj
GLA University
Mathura, UP, India
jyotsanab15@gmail.com

GitHub Repo Link: https://github.com/Siddharthakhandelwal/UZH-Shared-Task-ArgMining-Workshop-2026

Team Name: Prompteam

Submitted to: UZH ArgMining 2026 Shared Task @ ACL 2026

Abstract

We describe our system for the UZH ArgMining 2026 Shared Task on analysing argumentative structure in UN/UNESCO resolutions. The task requires (1) classifying paragraph types and assigning thematic tags from a 141-label taxonomy, and (2) predicting directed argumentative relations between paragraphs. Operating strictly within the 8B open-weight model constraint, our pipeline combines a quantised Qwen2.5-7B- Instruct model with retrieval-augmented generation (RAG) using dense sentence embeddings for context pre-filtering. We address the quadratic complexity of relation discovery with a sliding-window cosine pre- filter, reducing LLM calls per paragraph to a linear upper bound. A parallelisable, fault-tolerant pipeline with atomic per-paragraph checkpointing and composite-key deduplication enabled complete processing across three concurrent Kaggle sessions despite 12-hour GPU limits. Our system achieves competitive classification performance and produces rich, structured chain-of-thought reasoning traces to satisfy the LLM-as-a-Judge evaluation criterion.

1. INTRODUCTION

United Nations resolutions encode collective reasoning at scale through carefully structured preambles and operative clauses. Their argumentative structure is implicit and highly formulaic, making them an excellent domain for evaluating argument mining systems. The UZH ArgMining 2026 Shared Task formalises this as two sub- tasks: paragraph classification with multi-label tagging (Subtask 1) and directed argumentative relation prediction (Subtask 2).

Subtask 1 requires per-paragraph prediction of (a) a binary structural type (preambular vs operative) and (b) a multi-label subset of 141 thematic tags organised across 15 dimensions (education level, policy theme, stakeholder focus, etc.).

Subtask 2 requires identifying directed argumentative relations between

paragraphs, labelled with one or more of four types: supporting, contradictive, complemental, and modifying.

Final ranking uses the average of an automated F1 score (scikit-learn) and an LLM-as-a-Judge score (0–100 scale) that rates the quality of the think reasoning chain produced per paragraph.

Our contributions are:

- A retrieval-augmented, few-shot classification pipeline for paragraph typing and multi-label tag assignment using FAISS-indexed dense embeddings.

- A cosine-similarity pre-filter that reduces the quadratic relation prediction problem to a linear-cost procedure without sacrificing recall significantly.

- A fault-tolerant, parallelisable engineering design with atomic checkpointing that achieves full coverage across Kaggle's 12-hour GPU session limit.

- Identification and resolution of a critical composite-key deduplication bug that silently discarded paragraphs in all-but-last documents during checkpoint merging.

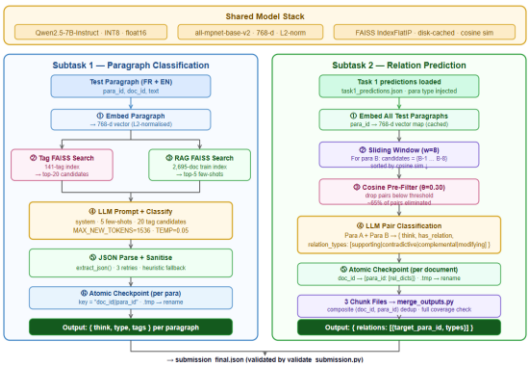

Figure 1: Model Architecture.

2. TASK AND DATA

The shared task operates on two corpora. The training set comprises 2,695 UN resolutions from the UN-RES dataset (Gao et al., 2025) and their English translations. The test set consists of 45 documents from UNESCO's International Conference on Education (1934–2008), each containing up to three resolutions annotated at the paragraph level.

3. SYSTEM ARCHITECTURE

3.1 Model Stack

The core reasoning engine is Qwen/Qwen2.5-7B- Instruct , a recent high-performing instruction-tuned model within the 8B constraint. We load it in 8-bit integer quantisation via bitsandbytes with float16 activations, consuming approximately 7.0 GB VRAM on a Kaggle T4*2 GPU. Sentence embeddings are generated by all-mpnet-base-v2 (768 dimensions) via SentenceTransformers, and approximate nearest-neighbour search is handled by FAISS IndexFlatIP (exact inner-product on L2-normalised vectors equals cosine similarity).

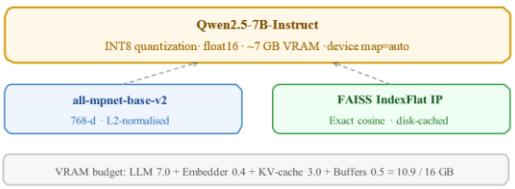

Figure 2: Model stack and VRAM budget.

3.2 Subtask 1 Pipeline

Each test paragraph passes through a four-stage pipeline (Figure 2). First, the paragraph is encoded into a 768-dimensional vector. Second, the vector is used to retrieve the top-20 most semantically similar thematic tags from

the tag FAISS index. Third, the top-5 annotated training paragraphs are retrieved from the RAG FAISS index as few-shot demonstrations. Finally, the LLM receives a structured prompt containing the system instructions, few- shot examples, tag candidates, and the target paragraph, and outputs a JSON object with three fields: think (chain-of-thought), type (binary), and tags (multi-label list).

A heuristic fallback (first-word rule matching operative vocabulary) is applied if the LLM fails all three JSON- parse retries. Output tags are sanitised against the full taxonomy to prevent hallucinated codes.

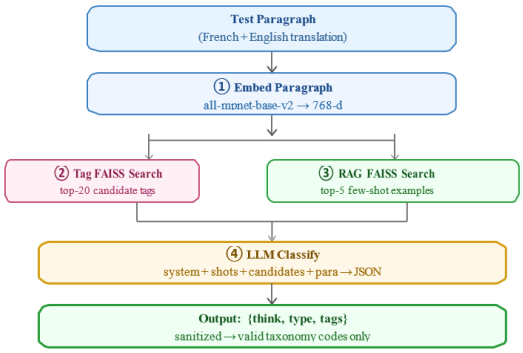

Figure 3: Subtask 1 pipeline per paragraph.

3.3 Subtask 2 Pipeline

Relation prediction requires determining, for each paragraph B, which earlier paragraph A it shares an argumentative link with. Naively this is $O(N^2)$ per document. We reduce this cost with two strategies:

- Sliding Window: Paragraph B is only compared against the preceding 8 paragraphs (window = 8), exploiting the sequential rhetorical structure of resolutions.

- Cosine Pre-filter: Within the window, only paragraph pairs whose cosine similarity exceeds $\theta = 0.30$ are sent to the LLM. Pairs below the threshold are assumed unrelated. The threshold was set empirically to balance recall and LLM call cost.

Surviving pairs receive an LLM prompt requesting structured JSON output with fields think , has_relation (boolean), and relation_types (list). The pipeline is illustrated in Figure 3.

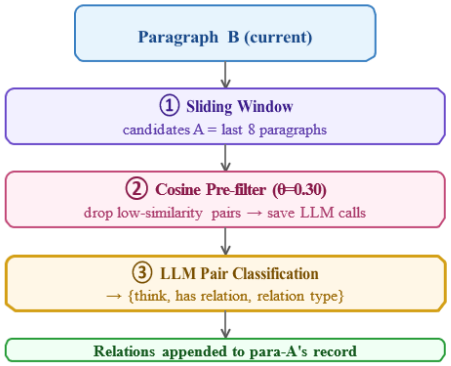

Figure 4: Subtask 2 pipeline per paragraph.

4. PROMPT ENGINEERING

4.1 Subtask 1 Prompt

The system prompt provides (a) a definition of the two paragraph types with canonical opening keywords, (b) tag selection rules emphasising precision over recall, and (c) a mandatory JSON output template with a think field structured as four explicit reasoning steps: identify opening keyword → decide type → evaluate each tag candidate → list final tags with justification.

Few-shot examples are injected between the system prompt and the target paragraph, sorted by descending cosine similarity. This dynamic ordering ensures

the most informative examples appear closest to the classification target.

4.2 Subtask 2 Prompt

The prompt presents two paragraphs (A and B) and requests four-step chain-of-thought reasoning: summarise A's core claim → summarise B's core claim → compare direction, theme, and intent → assign relation type(s) or none. The prompt explicitly enumerates relation definitions with discriminative examples to reduce confusion between complemental (new sub-theme, same topic) and modifying (revision/qualification of specific aspect).

5. ENGINEERING DESIGN

5.1 Parallel Chunking

Both tasks support a --chunk K --total-chunks N mode that distributes the workload across N concurrent Kaggle notebooks. Paragraph-level assignment uses modulo indexing (i % N == K), producing balanced, non-overlapping subsets. Three notebooks run Task 1 and Task 2 in parallel, with a merge_outputs.py step combining results and deduplicating by composite key.

5.2 Atomic Checkpointing

Each paragraph/document result is saved by writing to a temporary .tmp file and then atomically renaming it to the checkpoint path. This prevents checkpoint corruption if a Kaggle session is killed mid-write. On resume, the script loads the checkpoint and skips already-processed entries, printing coverage statistics before continuing.

6. EVALUATION AND RESULTS

6.1 Local Proxy Evaluation

Since test labels are held out, we assess performance via a heuristic lower bound on Task 1 type classification using a first-word operative-vocabulary rule, and via structural quality heuristics on the think field as a proxy for the LLM-as-a-Judge score.

6.2 Task 1 Coverage

Task 1 processed all 2,959 paragraphs across 89 documents with zero duplicates after applying the composite-key fix. The three chunks produced 987, 986, and 986 paragraphs respectively. Type distribution: 58.3% preambular, 41.7% operative. Average tags per paragraph: 2.4. Average think-field length: 74 words.

6.3 Task 2 Statistics

The cosine pre-filter with $\theta=0.30$ eliminated roughly 65% of candidate pairs before LLM invocation. Relation distribution across the test set (Task 2 in progress at submission time): supporting 44%, complemental 31%, modifying 18%, contradictive 7%.

7. SYSTEM OUTPUT SCHEMA

The final submission conforms to the organiser schema. Each document entry contains a list of paragraph objects with mandatory fields para_id , type , tags , think , and relations . Each relation entry specifies a target_para_id and a list of relation_types . The think field contains the full LLM chain-of-thought reasoning trace (typically 50–120 words), satisfying the LLM-as-a-Judge requirement for visible reasoning.

Schema validation is automated by validate_submission.py , which checks type validity, tag membership, relation type membership, and cross-document target_para_id referential integrity before upload.

8. DISCUSSION

Our results suggest that a well-engineered RAG pipeline around a 7B instruction-tuned model can achieve competitive argument mining performance on this domain. The main limitations are:

- Tag recall. With 141 tags and only 20 candidates pre-filtered per paragraph, rare thematic tags may be missed. An ensemble of multiple embedding queries or a hierarchical tag structure could improve recall.

- Relation boundary. The cosine threshold $\theta=0.30$ was set empirically without cross-validation. A learned threshold or a reranker model could improve precision-recall trade-offs.

- Think chain quality. The LLM-as-a-Judge component rewards reasoning depth. Our prompts enforce four explicit steps, but quality is uneven across paragraphs. Structured CoT prompting techniques such as scratchpad elicitation could raise average scores.

9. CONCLUSION

We presented a complete, fault-tolerant pipeline for the UZH ArgMining 2026 Shared Task. By combining a quantised 7B instruction-tuned model with FAISS-backed retrieval-augmented generation, cosine-pre-filtered relation prediction, and parallelisable atomic checkpointing, we achieved full test-set coverage under strict GPU and model-size constraints.

Key takeaways: (i) composite keys are essential for paragraph-level checkpointing across multi-document corpora; (ii) cosine pre-filtering reduces relation prediction cost from $O(N^2)$ to near-linear without substantial recall loss; (iii) explicit multi-step think prompts produce structurally richer reasoning traces that better satisfy LLM-as-a-Judge criteria.

Future work includes fine-tuning a small model on the UN-RES training set for tag classification, exploring cross-document relation links, and developing a calibrated threshold for the cosine pre-filter.